LAPORAN KEGIATAN UJI KOMPETENSI KEAHLIAN APLIKASI PENGEMBANGAN SISTEM PARKIR PINTAR BERBASIS INTERNET OF THINGS

Disusun oleh :

Nama : Ayla Pratiwi Mulyakin

NIS : 2324.10.353

PROGRAM KEAHLIAN PENGEMBAGAN PERANGKAT LUNAK DAN GIM

KOMPETENSI KEAHLIAN REKAYASA PERANGKAT LUNAK SMK MAHARDHIKA BATUAJAJAR

2025/2026

KOMPOTENSI KEAHLIAN : Rekayasa Perangkat Lunak

KELAS : XII RPL 1

NAMA LENGKAP : Ayla Pratiwi Mulyakin

NIS : 2324. 10. 353

PAKET SOAL : 2

JENIS APLIKASI : Website

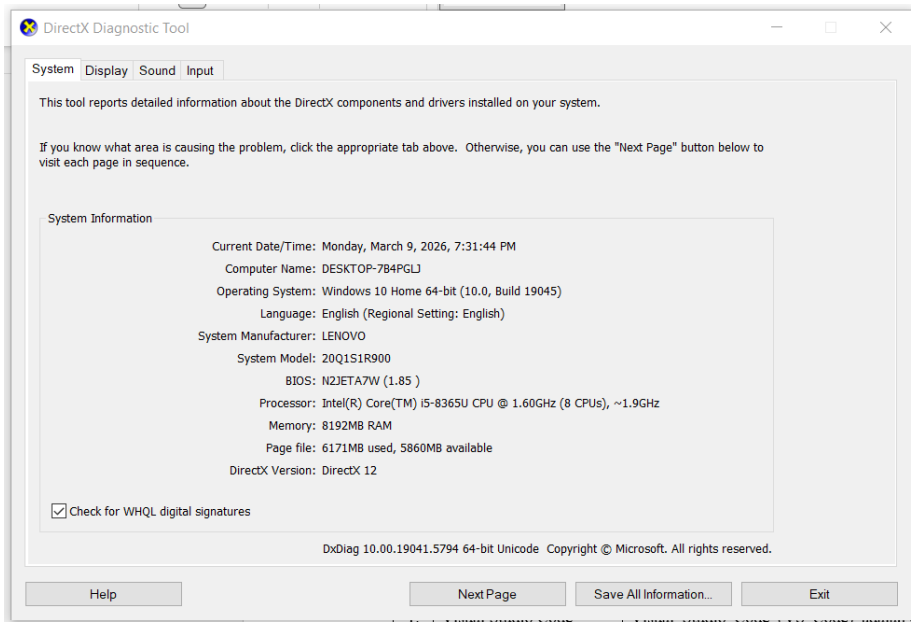
ALOKASI WAKTU : 12 JAM

---

1. PENJADWALAN KEGIATAN

Kegiatan	Alokasi																
	Minggu-1		Minggu-2			Minggu-3			Minggu-4			Minggu-5			Minggu-6		
	1	2	1	2	3	1	2	3	1	2	3	1	2	3	1	2	3
Analisis Soal																	
Perancangan Alur Sistem																	
Perancangan UI/UX																	
Perancangan Database																	
Implementasi Database																	
Implementasi UI/UX																	
Implementasi kode progam																	
Pengujian Aplikasi																	
Dokumen UG																	
Dokumentasi																	

2. SPESIFIKASI KOMPUTER YANG DI GUNAKAN



3. PERSIAPAN PERANGKAT KERAS

Berikut merupakan perangkat keras device Internet Of Things yang digunakan dalam pengerjaan.

1. **ESP32**

ESP32 digunakan sebagai pusat kendali (controller) pada system parkir berbasis IOT.Perangkat ini berfungsi untuk menghubungkan seluruh komponen seperti RFID, Servo, dan LCD dengan server melalui jaringan WIFI. ESP32 menerima data dari RFID, mengirimkannya ke server untuk diproses, lalu mengeksekusi perintah seperti membuka palang dan menampilkan informasi pada LCD.

2. **SERVO ENTRY**

Servo Entry digunakan sebagai aktuator untuk membuka dan menutup palang pintu masuk. Setelah server memvalidasi kartu RFID, ESP32 memberikan perintah kepada servo untuk berputar dan membuka palang selama beberapa detik, kemudian menutup kembali secara otomatis**.**

3. **RFID ENTRY**

RFID Entry digunakan sebagai alat identifikasi kendaraan saat memasuki area parkir. Ketika kartu ditempelkan, modul RFID membaca ID kartu dan mengirimkan data tersebut ke ESP32. Selanjutnya, data dikirim ke server untuk dicatat sebagai waktu masuk (Check-In) dan status kendaraan diubah menjadi "IN".

4. **RFID EXIT**

RFID Exit digunakan untuk mengidentifikasi kendaraan yang akan keluar dari area parkir. Saat kartu ditempelkan, sistem akan membaca ID kartu dan mengirimkannya ke server untuk menghitung durasi parkir serta biaya secara real-time. Setelah proses validasi dan pembayaran selesai, status kendaraan diubah sesuai proses transaksi.

5. **SERVO EXIT**

Servo Exit berfungsi untuk membuka palang pintu keluar setelah sistem memastikan bahwa kendaraan telah menyelesaikan proses parkir dan pembayaran. Perintah membuka palang dikirim oleh ESP32 berdasarkan respon dari server**.**

6. **LCD**

LCD digunakan sebagai media tampilan informasi kepada pengguna parkir. Layar ini menampilkan pesan seperti status kartu, durasi parkir, biaya yang harus dibayar, serta notifikasi ketika palang terbuka atau terjadi kesalahan. LCD membantu memberikan umpan balik secara langsung kepada pengguna sistem.

4. PERSIAPAN PERANGKAT LUNAK

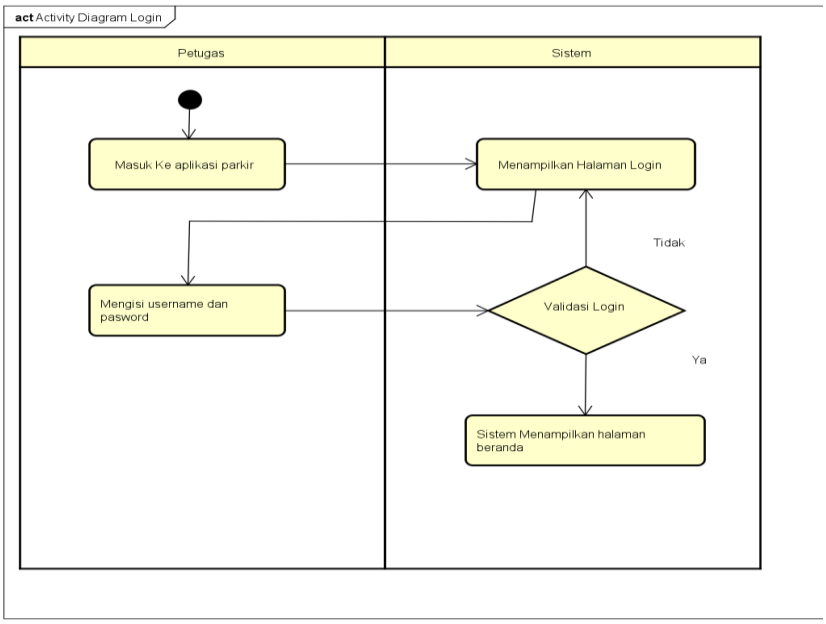
No	Aplikasi Perangkat Lunak	Pengertian
1.	Visual Studio Code	Editor kode sumber yang dikembangkan oleh Microsoft untuk menulis program dengan berbagai bahasa pemrograman seperti JavaScript, Python, PHP, dan lainnya. VSCode dilengkapi fitur IntelliSense, debugging, dan ekstensi yang dapat ditambahkan sesuai kebutuhan.

No	Aplikasi Perangkat Lunak	Pengertian
2.	Xampp	Paket software web server open-source yang berisi Apache, MySQL/MariaDB, PHP, dan Perl. Berfungsi untuk membuat lingkungan server lokal di komputer sendiri sehingga bisa mengembangkan dan menguji aplikasi web sebelum di-upload ke hosting.
3.	Astah Professional	Software modeling yang digunakan untuk membuat berbagai diagram dalam perancangan sistem seperti UML, ERD, DFD, dan flowchart. Software ini membantu dalam mendokumentasikan dan memvisualisasikan desain sistem secara profesional.
4.	Balsamiq	Software wireframing untuk membuat desain antarmuka (UI) aplikasi atau website secara cepat. Berguna untuk membuat sketsa tampilan aplikasi sebelum masuk ke tahap desain visual yang lebih detail.
5.	Microsoft Word	Aplikasi pengolah kata dari Microsoft untuk membuat, mengedit, dan memformat dokumen teks. Digunakan untuk membuat surat, laporan, makalah, skripsi, dan berbagai jenis dokumen dengan fitur formatting yang lengkap.
6.	Google chrome	Browser web yang dikembangkan oleh Google untuk mengakses internet dan menjelajahi website. Chrome dikenal cepat, aman, dan mendukung sinkronisasi data dengan akun Google di berbagai perangkat.
7	MQTTX	Aplikasi client MQTT yang digunakan untuk mengirim, menerima, dan menguji pesan pada broker MQTT secara mudah melalui tampilan antarmuka.

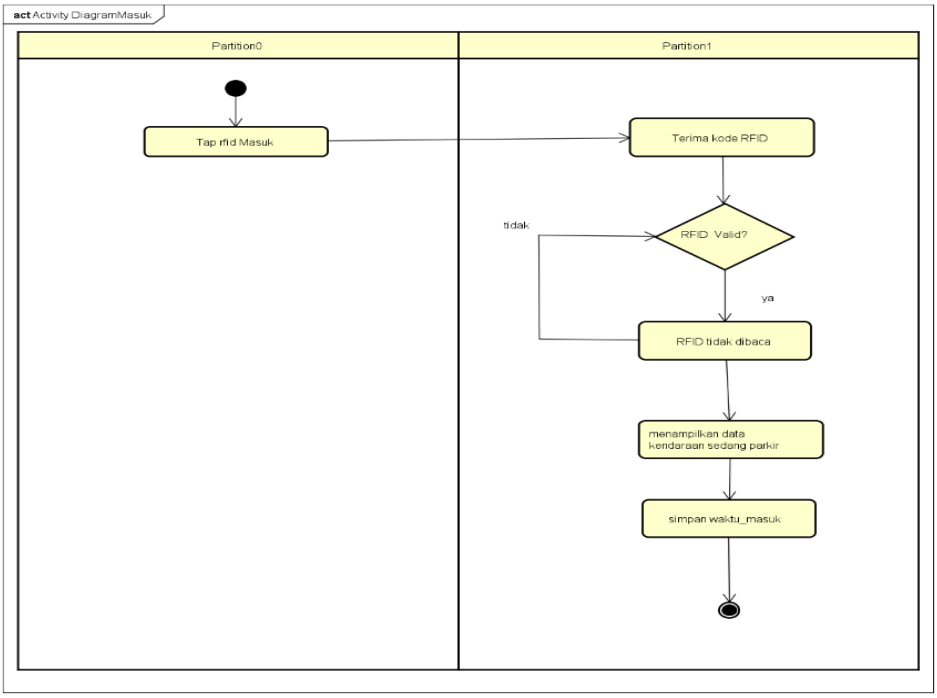
No	Aplikasi Perangkat Lunak	Pengertian
8.	Arduino IDE	Software untuk menulis, mengedit, dan mengunggah program ke papan Arduino. Digunakan untuk mengembangkan proyek berbasis mikrokontroler
9.	HiveMq	Platform MQTT broker yang digunakan untuk mengirim dan menerima data antar perangkat secara real-time, sering dipakai pada sistem IoT.
10.	Canva	Aplikasi desain grafis berbasis web yang digunakan untuk membuat poster, presentasi, dan berbagai desain visual dengan mudah.
11.	Github	Platform untuk menyimpan dan mengelola kode program menggunakan Git, serta mendukung kolaborasi dalam pengembangan software.

5. PERANCANGAAN ALGORITMA APLIKASI

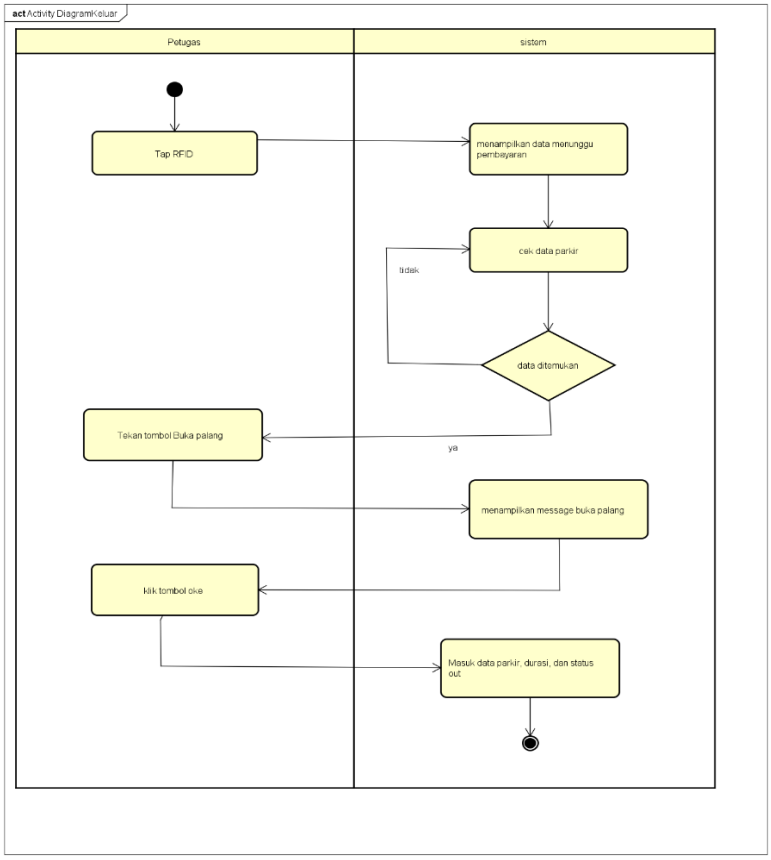
Berikut merupakan perancangan algoritma aplikasi yang digambarkan dalam bentuk diagram.



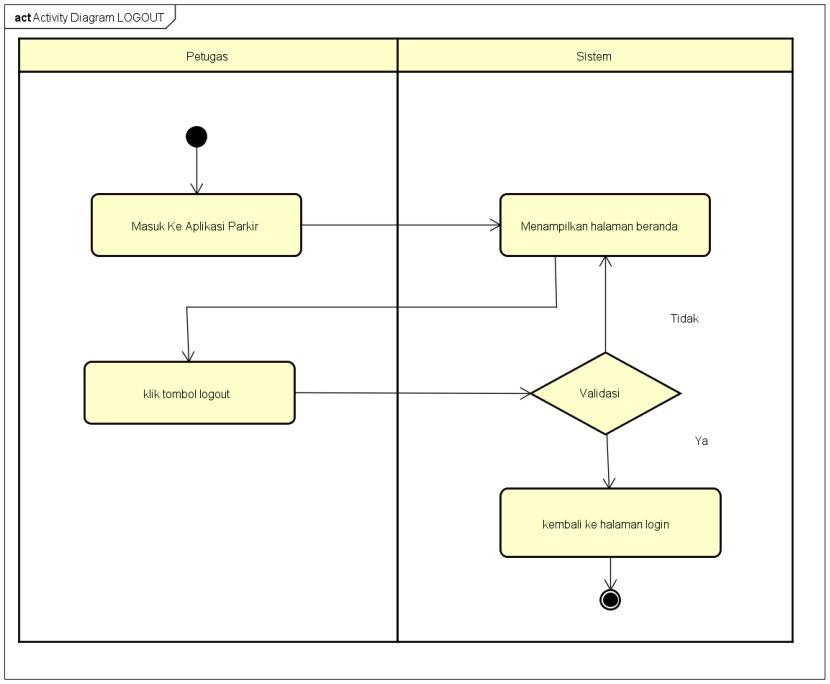
Gambar 1. 1 Activty diagram login



Gambar 1. 2 Activty diagram Masuk



Gambar 1. 3 Activty diagram Keluar

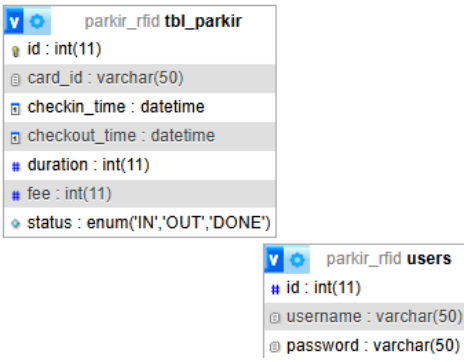


Gambar 1. 4 Activity diagram logout

6. PERANCANGAN DESAIN BASIS DATA

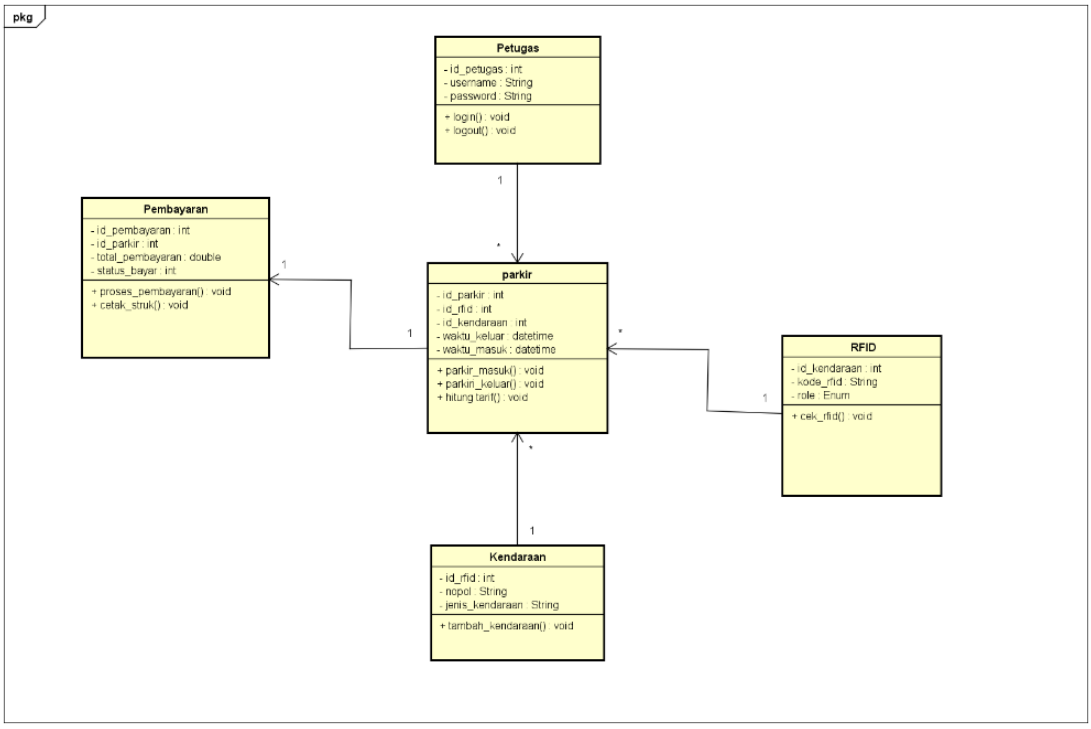
Berikut merupakan desain basis data pada *phpMyAdmin* beserta penggambaran visual nya dalam bentuk diagram **Class diagram dan ERD (**Entity Relationship Diagram)

1. **Desain Basis Data**



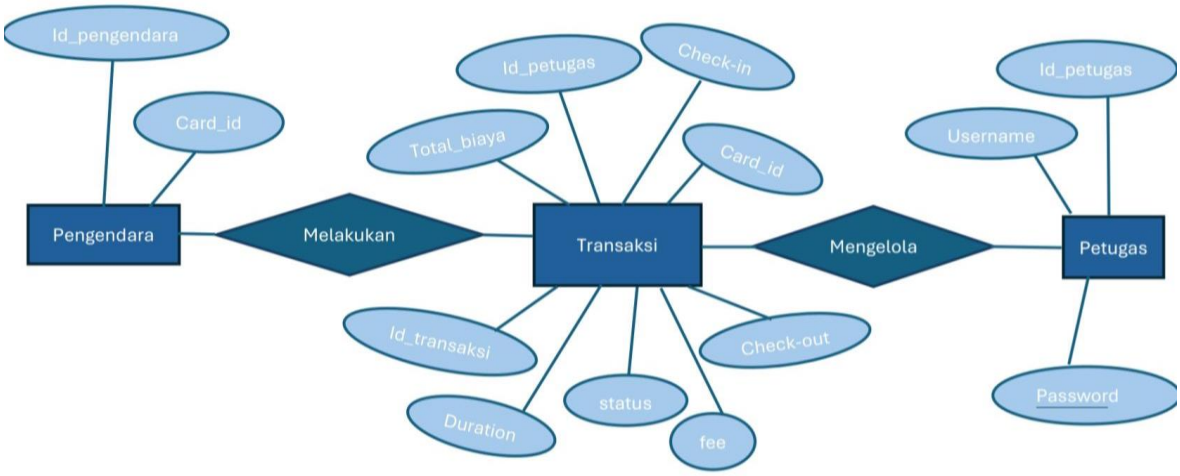
Gambar 1. 4 Desain Basis Data

2. **Class Diagram**



Gambar 1. 5 Class diagram

3. **ERD (**Entity Relationship Diagram)



Gambar 1. 6 Entity Relationship Diagram (ERD)

7. TABEL BASIS DATA

Berikut merupakan daftar tabel pada basis data beserta dengan struktur masing masing tabel-nya.

1. **Tabel**

users

id	username	password	role
3	petugas	…	petugas

tbl_Parkir

id	Card_id	Checkin_time	Checkout_time	Duration	Fee	Status
1	1234	2026-03-28 07:56:08	2026-03-28 13:56:08	1jam	2000	DONE
2	1234	2026-03-28 13:29:51	2026-03-28 13:30:06	1jam	2000	DONE

2. **Struktur Tabel**

Tabel_users

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**id**	int(11)			No	*None*			Change Drop More
2	**username**	varchar(50)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More
3	**password**	varchar(50)	utf8mb4_general_ci		Yes	*NULL*			Change Drop More

Gambar 1. 7 Database tabel_users

Tabel_Parkir

Table structure | Relation view

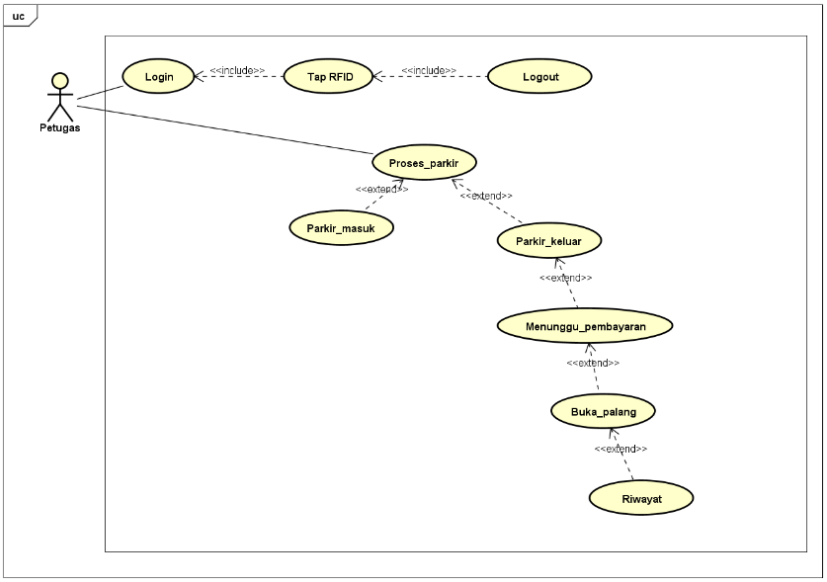
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**id**	int(11)			No	*None*		AUTO_INCREMENT	Change Drop More
2	**card_id**	varchar(50)	utf8mb4_general_ci		No	*None*			Change Drop More
3	**checkin_time**	datetime			No	*None*			Change Drop More
4	**checkout_time**	datetime			Yes	*NULL*			Change Drop More
5	**duration**	int(11)			Yes	*NULL*			Change Drop More
6	**fee**	int(11)			Yes	*NULL*			Change Drop More
7	**status**	enum('IN', 'OUT', 'DONE')	utf8mb4_general_ci		No	IN			Change Drop More

Gambar 1. 8 Database tabel_parkir

8. PERANCANGAN TAMPILAN ANATARMUKA (UI) APLIKASI

Berikut merupakan penggambaran visual dari fitur yang terdapat pada aplikasi dalam bentuk diagram.

a. **UseCase Diagram**



Gambar 1. 9 Use Case Skenario

b. **Use Case Skenario**

1. Deskripsi Actor

No	Aktor	Deskripsi
1.	User	Petugas adalah pengguna sistem yang bertugas mengelola parkir, seperti memantau kendaraan masuk dan keluar, memproses pembayaran, serta membuka palang parkir melalui sistem dashboard.

2. Use Case Skenario

Name : Login
IDE : UC001
Descripsion : Petugas harus login terlebih dahulu untuk mengakses sistem dashboard parkir.
Actors : Petugas

Main Course	
Aksi Aktor	Reaksi Sistem
1. Membuka halaman login	
	2. Menampilkan form login
3. Mengisi username dan password	
	4. Sistem melakukan validasi data
	5. Menampilkan halaman dashboard
Alternate Course	

Exception	
Aksi Aktor	Reaksi Sistem
1. Mengisi data login	
	2. Data tidak sesuai
	3. Sistem menampilkan pesan "Login gagal"

Name : Melihat Data Parkir
IDE : UC002
Descripsion : Petugas melihat data kendaraan yang sedang parkir, menunggu pembayaran,& Riwayat transaksi
Actors : Petugas

Main Course	
Aksi Aktor	Reaksi Sistem
1. Membuka dashboard	
	2. Menampilkan halaman dashboard
3. Melihat data kendaraan masuk	
	4. Menampilkan data kendaraan dengan status IN
5. Melihat kendaraan keluar	
	6. Menampilkan data kendaraan status OUT (menunggu bayar)
7. Melihat riwayat transaksi	
	8. Menampilkan data riwayat parkir
Alternate Course	

Exception	
Aksi Aktor	Reaksi Sistem
1. Membuka dashboard	
	2. Tidak ada data
	3. Sistem menampilkan pesan "Tidak ada data"

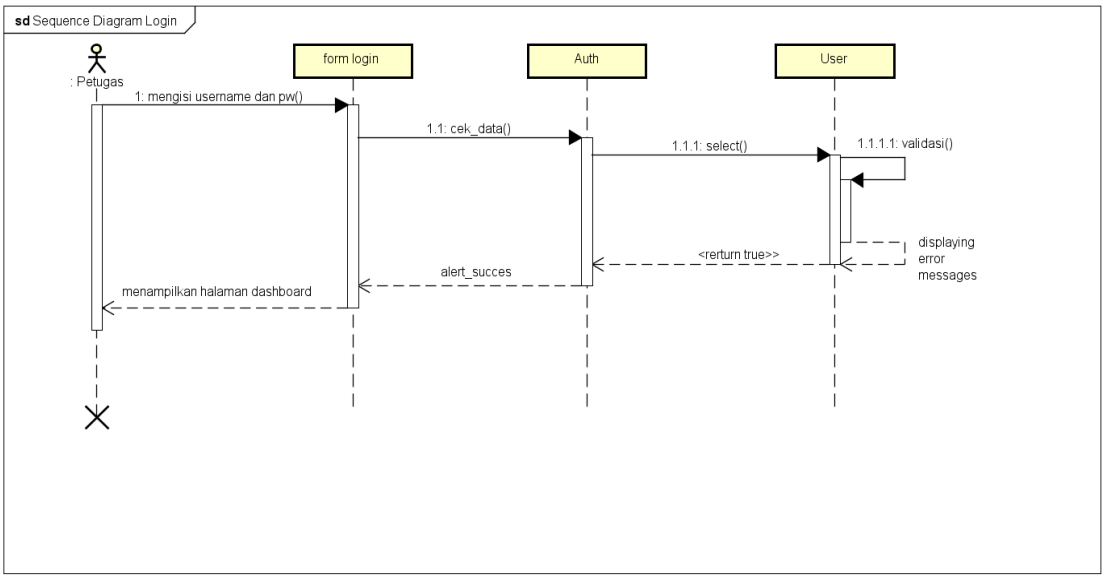
Name : Buka Palang
IDE : UC003
Descripsion : Petugas membuka palang keluar setelah kendaraan melakukan pembayaran..
Actors : Petugas

Main Course	
1. Memilih kendaraan yang akan keluar	
	2. Menampilkan data kendaraan dan biaya
3. Menekan tombol "Buka Palang"	
	4. Sistem memproses pembayaran
	5. Sistem mengubah status menjadi OUT
	6. Sistem mengirim perintah MQTT ke servo
	7. Palang terbuka
	8. Sistem menampilkan notifikasi berhasil
Alternate Course	

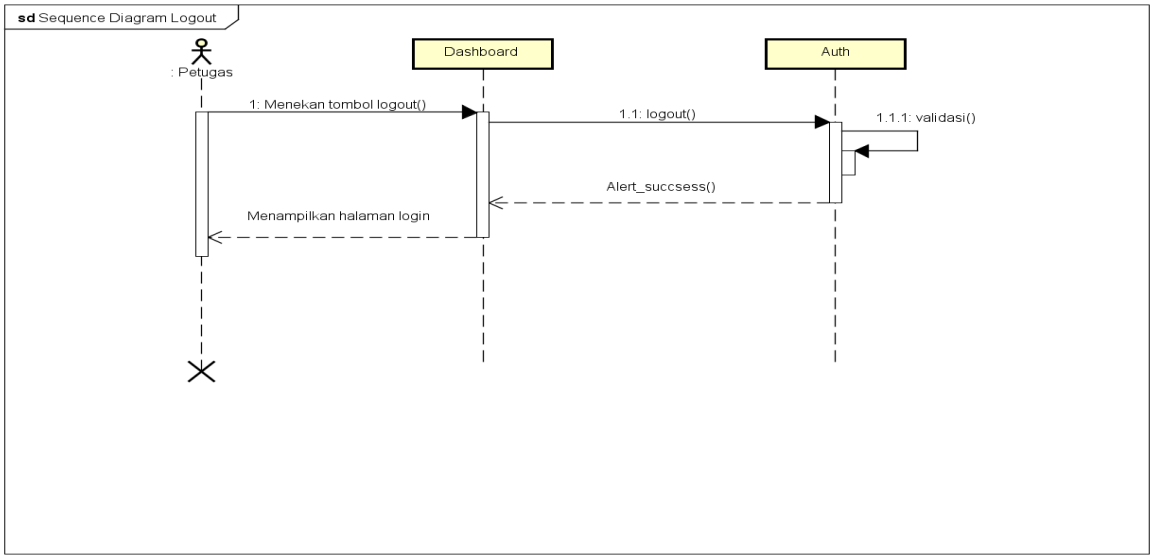
Exception	
Aksi Aktor	Reaksi Sistem
1. Menekan tombol buka palang	
	2. Koneksi MQTT gagal
	3. Sistem menampilkan pesan error
4. Menekan tombol buka palang	
	5. Data tidak ditemukan
	6. Sistem menampilkan pesan "Data tidak valid"

9. PERANCANGAN OBJEK DALAM SISTEM

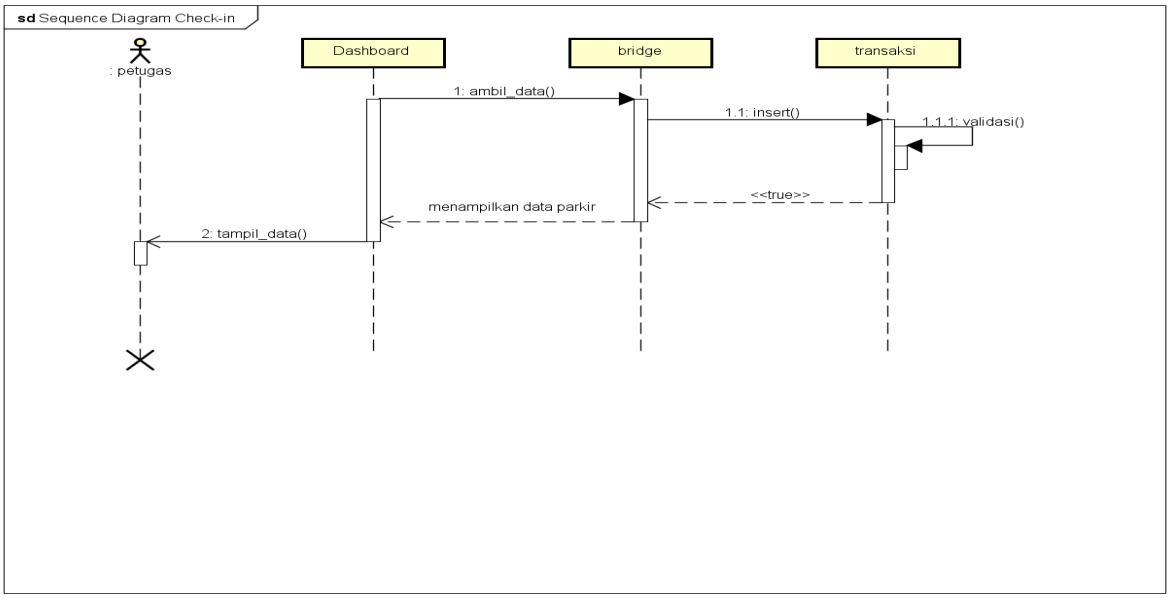
Berikut merupakan penggambaran visual dari urutan interaksi dinamis antar objek dalam sistem secara kronologis dalam bentuk diagram **Sequence Diagram**



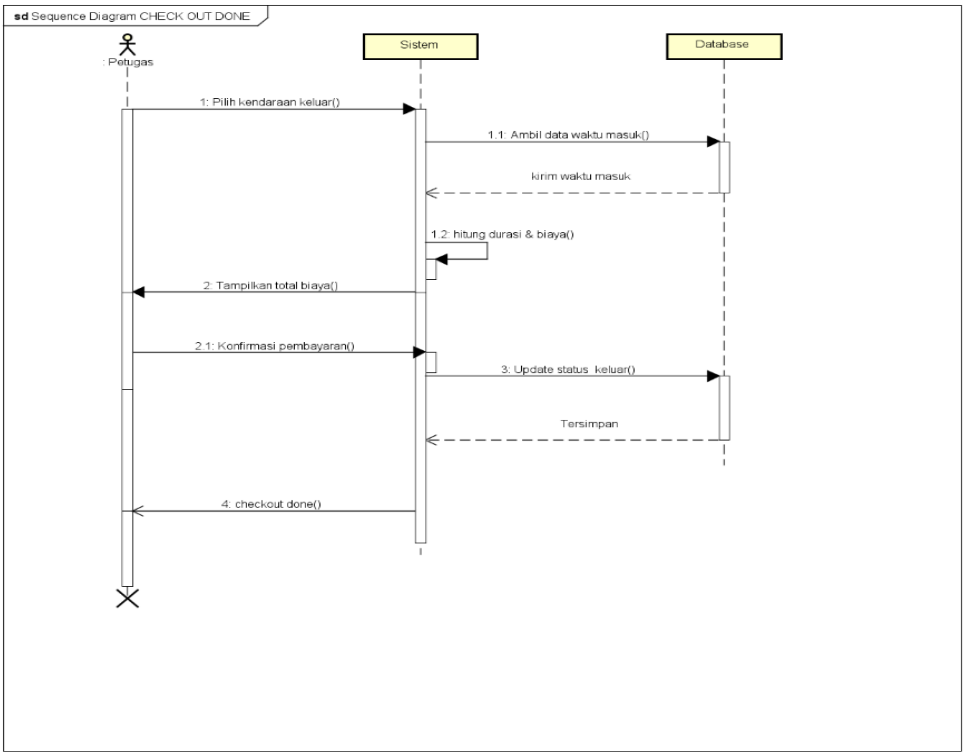
Gambar 1. 10 Sequence Diagram Login



Gambar 1. 11 Sequence Diagram Logout



Gambar 1. 12 Sequence Diagram Check-in



Gambar 1. 13 Sequence Diagram Check-out Done

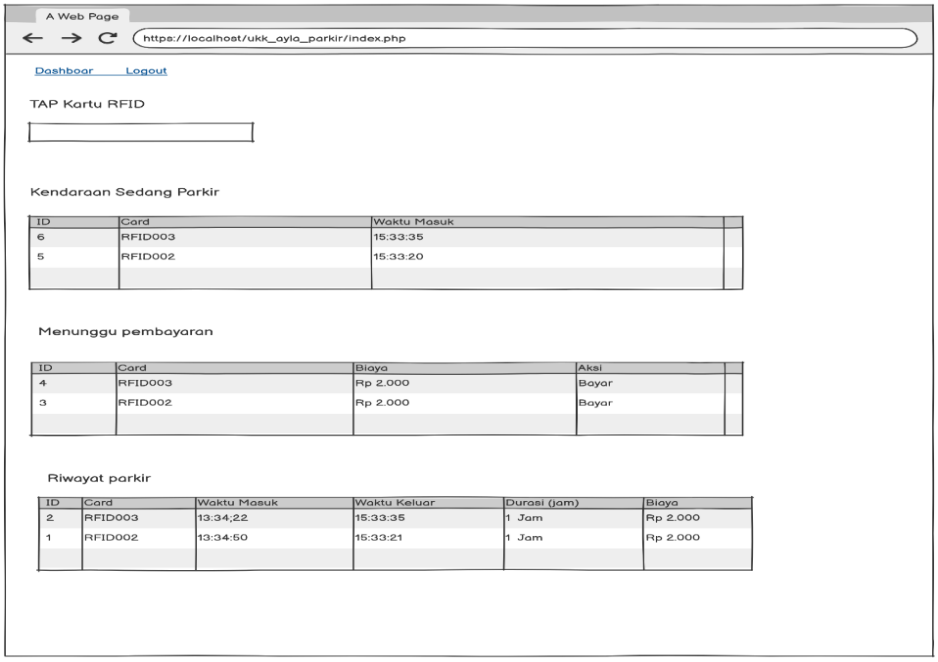
10. MOCK UP APLIKASI

Berikut merupakan beberapa Mockup yang digunakan sebagai representasi visual realistis dari aplikasi yang dibuat.

1. Tampilan Login

Gambar 1. 14 Mock-up Login

2. Dashboard Petugas

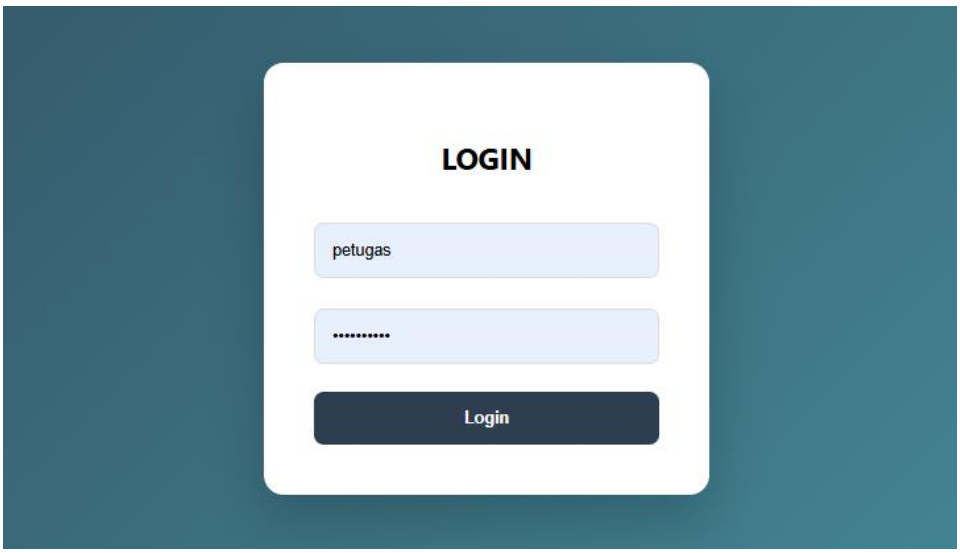


Gambar 1. 15 Mock-up Dashboard

11. IMPLEMENTASI TAMPILAN ANTARMUKA (UI)

Berikut merupakan implementasi dari tampilan antarmuka (UI) dari aplikasi parkir.

1. Tampilan Login



Gambar 1. 16 Implementasi Tampilan Login

Halaman login merupakan antarmuka awal yang digunakan petugas untuk masuk ke sistem. Halaman ini memiliki komponen:

Komponen yang tersedia:

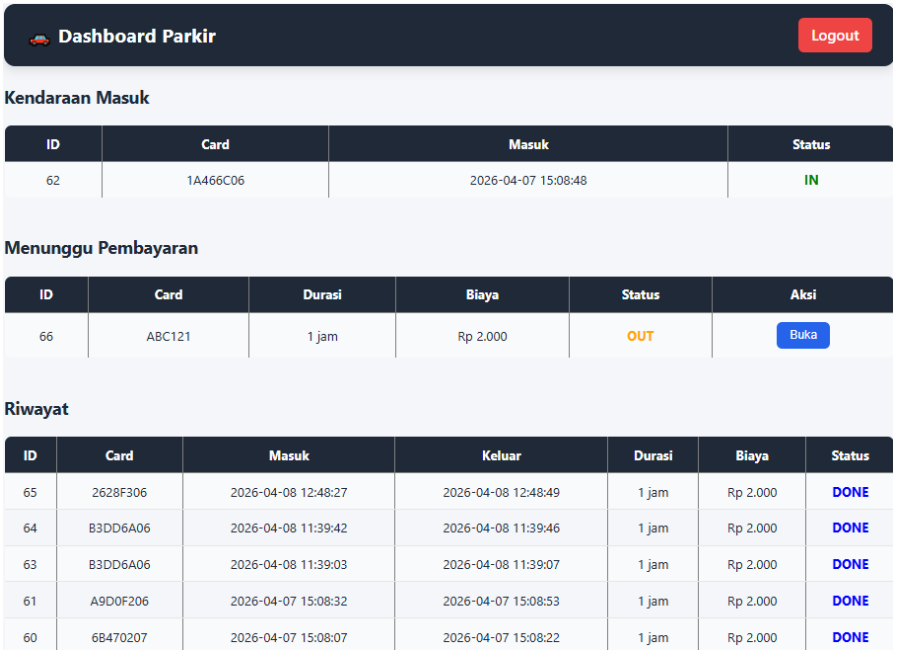
- Input Username: Kolom untuk memasukkan username petugas
- Input Password: Kolom untuk memasukkan password (tersamar)
- Tombol Login: Tombol untuDk memproses autentikasi
- Pesan Error: Menampilkan notifikasi jika login gagal

Cara penggunaan:

1. Petugas membuka aplikasi parkir di browser
2. Masukkan username "petugas" dan password yang sudah terdaftar
3. Klik tombol "Login"
4. Sistem akan memvalidasi data ke database
5. Jika berhasil, petugas diarahkan ke dashboard

Fungsi: Keamanan sistem agar hanya petugas yang berwenang yang dapat mengaksaplikasi parkir.

2. Dashboard Petugas



Gambar 1. 17 Implementasi Tampilan Dashboard

Dashboard adalah halaman utama yang menampilkan informasi real-time tentang aktivitas parkir. Dashboard terbagi menjadi 3 bagian:

1. Kendaraan Masuk (Status IN)

- Menampilkan kendaraan yang sedang parkir
- Informasi: ID, Card ID, dan waktu masuk
- Status ditampilkan dengan warna hijau

2. Menunggu Pembayaran (Status OUT)

- Menampilkan kendaraan yang sudah check-out dan menunggu pembayaran
- Informasi: ID, Card ID, durasi parkir, dan biaya
- Tombol "Buka": Untuk membuka palang setelah pembayaran dikonfirmasi

3. Riwayat Transaksi (Status DONE)

- Menampilkan 50 transaksi terakhir yang sudah selesai
- Informasi lengkap: waktu masuk, keluar, durasi, dan biaya

12. PEMBUATAN STYLE WEB (CSS)

Berikut merupakan kode **CSS** (Cascading Style Sheet) yang digunakan untuk mengubah dan menambahkan, baik teks, gambar,hingga latar belakang sebuah halaman aplikasi *website*.

1. **style.css**

```
/* ================= GLOBAL ================= */
body{
    font-family: 'Segoe UI', Arial, sans-serif;
    background:#f4f6f9;
    margin:0;
    padding:20px;
    color:#1f2937;
}

/* ================= NAVBAR ================= */
.navbar{
    background:#1f2937;
    padding:15px 25px;
    border-radius:12px;
```

```
        margin-bottom:20px;
        display:flex;
        justify-content:space-between;
        align-items:center;
        box-shadow:0 4px 10px rgba(0,0,0,0.1);
    }

    .navbar h1{
        color:#ffffff;
        margin:0;
        font-size:20px;
        letter-spacing:0.5px;
    }

    .navbar a{
        color:white;
        font-weight:500;
        text-decoration:none;
        background:#ef4444;
        padding:8px 14px;
        border-radius:6px;
        transition:all 0.3s ease;
    }

    .navbar a:hover{
        background:#dc2626;
        transform:scale(1.05);
    }

    /* ================= SECTION ================= */
    section{
        background:white;
        padding:18px;
        border-radius:12px;
```

```
        box-shadow:0 4px 12px rgba(0,0,0,0.08);
        margin-bottom:20px;
        transition:0.3s;
    }

    section:hover{
        transform:translateY(-2px);
    }

    /* ================= TITLE ================= */
    h2{
        color:#1f2937;
        margin-bottom:12px;
        font-size:18px;
    }

    /* ================= TABLE ================= */
    table{
        width:100%;
        border-collapse:collapse;
        margin-top:10px;
        overflow:hidden;
        border-radius:8px;
    }

    table th{
        background:#1f2937;
        color:white;
        padding:10px;
        font-size:14px;
    }

    table td{
        padding:10px;
```

```
        text-align:center;
        border-bottom:1px solid #e5e7eb;
        font-size:14px;
    }

    table tr:nth-child(even){
        background:#f9fafb;
    }

    table tr:hover{
        background:#e5e7eb;
        transition:0.2s;
    }

    /* ================= BUTTON ================= */
    button{
        padding:7px 14px;
        background:#2563eb;
        color:white;
        border:none;
        cursor:pointer;
        border-radius:6px;
        transition:all 0.3s ease;
        font-size:13px;
    }

    button:hover{
        background:#1d4ed8;
        transform:scale(1.05);
    }

    /* ================= CETAK ================= */
    .btn-cetak{
        background:#16a34a;
```

```
        color:white;
        padding:7px 14px;
        border-radius:6px;
        text-decoration:none;
        margin-left:5px;
        font-size:13px;
        transition:all 0.3s ease;
    }

    .btn-cetak:hover{
        background:#15803d;
        transform:scale(1.05);
    }

    /* ================= STATUS ================= */
    .status-in{
        color:#16a34a;
        font-weight:bold;
    }

    .status-done{
        color:#2563eb;
        font-weight:bold;
    }

    .status-out{
        color:#f59e0b;
        font-weight:bold;
    }

    /* ================= LINE ================= */
    hr{
        border:none;
        height:1px;
```

```
        background:#e5e7eb;
        margin:25px 0;
    }
```

13. PEMBUATAN KODE WEB HYPERTEXT PREPOCESSOR (PHP)

Berikut merupakan kode PHP (Hypertext Prepocessor) yang digunakan untuk membangun situs web dinamis dan aplikasi web sisi server (Backend) dengan memungkinkan interaksi dengan *database,*memproses formulir,mengelola sesi pengguna dan menghasilkan konten yang merubah secara *real-time*.

1. login.php

```
<?php
session_start();
require_once 'config/koneksi.php';

if (isset($_POST['login'])) {

    $username = $_POST['username'];
    $password = $_POST['password'];

    $query = "SELECT * FROM users
              WHERE username='$username'
              AND password='$password'";

    $result = mysqli_query($conn, $query);

    if ($result && mysqli_num_rows($result) > 0) {

        $user = mysqli_fetch_assoc($result);

        $_SESSION['user'] = $user['username'];

        header("Location: index.php");
        exit;
```

```
        } else {
            $error = "Username atau Password salah!";
        }
    }
    ?>
    <!DOCTYPE html>
    <html>

    <head>
        <title>Login Sistem Parkir</title>
        <style>
            * {
                box-sizing: border-box;
            }

            body {
                font-family: 'Segoe UI', sans-serif;
                background: linear-gradient(135deg, #2c3e50,
    #4ca1af);
                display: flex;
                justify-content: center;
                align-items: center;
                height: 100vh;
                margin: 0;
            }

            .card {
                background: white;
                padding: 40px;
                width: 360px;
                border-radius: 16px;
                box-shadow: 0 20px 45px rgba(0, 0, 0, 0.2);
            }
```

```
h2 {
    text-align: center;
    margin-bottom: 25px;
}

input {
    width: 100%;
    padding: 14px;
    margin: 12px 0;
    border: 1px solid #ddd;
    border-radius: 8px;
}

button {
    width: 100%;
    padding: 14px;
    margin-top: 10px;
    background: #2c3e50;
    color: white;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    font-weight: bold;
}

button:hover {
    background: #1a252f;
}

.error {
    color: red;
    text-align: center;
    margin-bottom: 10px;
}
```

```
        </style>
    </head>

    <body>

        <div class="card">
            <h2>LOGIN</h2>

            <?php    if    (isset($error))    echo    "<div
    class='error'>$error</div>"; ?>

            <form method="POST">
                <input      type="text"      name="username"
    placeholder="Username" required>
                <input    type="password"    name="password"
    placeholder="Password" required>
                <button                        type="submit"
    name="login">Login</button>
            </form>

        </div>

    </body>

    </html>
```

2. index.php

```
<?php
session_start();

// ================= REQUIRE =================
require_once __DIR__ . '/config/koneksi.php';
require_once __DIR__ . '/models/ParkirModel.php';
require_once __DIR__ .
```

```
'/controllers/ParkirController.php';
require __DIR__ . '/vendor/autoload.php';

use PhpMqtt\Client\MqttClient;

// ================= INIT =================
$model = new ParkirModel();
$controller = new ParkirController();

if (!isset($_SESSION['user'])) {
header("Location: login.php");
exit;
}

//    =================    HANDLE    BUKA    PALANG
=================
if         (isset($_POST['buka_palang'])         &&
isset($_POST['id'])) {

$id = $_POST['id'];

// update status jadi DONE
$controller->bukaPalang($id);

// ================= MQTT =================
$config = require __DIR__ . '/config/mqtt.php';

try {

$mqtt = new MqttClient(
$config['broker'],
$config['port'],
'web-' . uniqid()
);
```

```
$mqtt->connect();

// buka palang exit
$mqtt->publish(
$config['prefix'].'/'.$config['topic_exit_servo'],
'OPEN',
1
);

// tampil OLED
$mqtt->publish(
$config['prefix'].'/'.$config['topic_lcd'],
'Terima Kasih|Selamat Jalan',
1
);

$mqtt->loop(true);
sleep(1);

$mqtt->disconnect();

} catch (Exception $e) {
echo "MQTT ERROR: " . $e->getMessage();
}

exit;
}
?>

<!DOCTYPE html>
<html>
<head>
<title>Dashboard Parkir</title>
```

```
<link rel="stylesheet" href="asset/style.css">
</head>

<body>

<div class="navbar">
<h2>🚗 Dashboard Parkir</h2>
<a href="logout.php">Logout</a>
</div>

<!--     =================     KENDARAAN     MASUK
================= -->
<h3>Kendaraan Masuk</h3>
<table border="1" width="100%">
<tr>
<th>ID</th>
<th>Card</th>
<th>Masuk</th>
<th>Status</th>
</tr>

<?php
$data                                                =
$model->getParkirAktif()->fetchAll(PDO::FETCH_ASSO
C);

if ($data) {
foreach ($data as $row) {
echo "<tr>
<td>{$row['id']}</td>
<td>{$row['card_id']}</td>
<td>{$row['checkin_time']}</td>
<td style='color:green;'>IN</td>
</tr>";
```

```
}
} else {
echo "<tr><td colspan='4'>Kosong</td></tr>";
}
?>
</table>

<br>

<!--    =================    MENUNGGU    PEMBAYARAN
================= -->
<h3>Menunggu Pembayaran</h3>
<table border="1" width="100%">
<tr>
<th>ID</th>
<th>Card</th>
<th>Durasi</th>
<th>Biaya</th>
<th>Aksi</th>
</tr>

<?php
$data                                                =
$model->getMenungguBayar()->fetchAll(PDO::FETCH_AS
SOC);

if ($data) {
foreach ($data as $row) {
echo "<tr>
<td>{$row['id']}</td>
<td>{$row['card_id']}</td>
<td>{$row['duration']} jam</td>
<td>Rp " . number_format($row['fee'], 0, ',', '.') .
"</td>
```

```
<td>
<button
onclick='bukaPalang({$row['id']})'>Buka</button>
</td>
</tr>";
}
} else {
echo "<tr><td colspan='5'>Kosong</td></tr>";
}
?>
</table>

<br>

<!-- ================= RIWAYAT ================= -
->
<h3>Riwayat</h3>
<table border="1" width="100%">
<tr>
<th>ID</th>
<th>Card</th>
<th>Masuk</th>
<th>Keluar</th>
<th>Durasi</th>
<th>Biaya</th>
</tr>

<?php
$data                                                =
$model->getHistory()->fetchAll(PDO::FETCH_ASSOC);

if ($data) {
foreach ($data as $row) {
echo "<tr>
```

```
<td>{$row['id']}</td>
<td>{$row['card_id']}</td>
<td>{$row['checkin_time']}</td>
<td>{$row['checkout_time']}</td>
<td>{$row['duration']} jam</td>
<td>Rp " . number_format($row['fee'], 0, ',', '.') .
"</td>
</tr>";
}
} else {
echo "<tr><td colspan='6'>Kosong</td></tr>";
}
?>
</table>

<!-- ================= SCRIPT ================= -
->
<script>

function bukaPalang(id) {

if (!confirm("Buka palang kendaraan ini?")) return;

let form = new FormData();
form.append("id", id);
form.append("buka_palang", "1");

fetch("index.php", {
method: "POST",
body: form
})
.then(() => {
alert("Palang terbuka!");
location.reload();
```

```
})
.catch(err => {
alert("Error: " + err);
});
}

// auto refresh tiap 5 detik
setInterval(() => location.reload(), 5000);

</script>

</body>
</html>
```

3. **Model/ParkirModel.php**

```
<?php
require_once __DIR__ . '/../config/koneksi.php';

class ParkirModel {
    private $conn;
    private $table = "tbl_parkir";
    private $tarif_per_jam = 2000;

    public function __construct() {
        global $conn;
        $this->conn = $conn;
    }

    // ================== AKTIF ==================
    public function getParkirAktif() {
        $stmt = $this->conn->prepare("
            SELECT * FROM {$this->table}
            WHERE status='IN'
            ORDER BY checkin_time DESC
```

```
                ");
                $stmt->execute();
                return $stmt;
            }

            //         ==================         MENUNGGU
        ==================
            public function getMenungguBayar() {
                $stmt = $this->conn->prepare("
                    SELECT * FROM {$this->table}
                    WHERE status='WAITING'
                    ORDER BY id DESC
                ");
                $stmt->execute();
                return $stmt;
            }

            //          ==================          HISTORY
        ==================
            public function getHistory() {
                $stmt = $this->conn->prepare("
                    SELECT * FROM {$this->table}
                    WHERE status='DONE'
                    ORDER BY id DESC LIMIT 50
                ");
                $stmt->execute();
                return $stmt;
            }

            // ================== BY ID ==================
            public function getParkirById($id) {
                $stmt = $this->conn->prepare("
                    SELECT * FROM {$this->table}
                    WHERE id = ?
```

```
        ");
        $stmt->execute([$id]);
        return $stmt->fetch(PDO::FETCH_ASSOC);
    }

    //      ==================      BY      CARD
==================
    public  function  getByCardAndStatus($card_id,
$status) {
        $stmt = $this->conn->prepare("
            SELECT * FROM {$this->table}
            WHERE card_id = ? AND status = ?
            ORDER BY id DESC LIMIT 1
        ");
        $stmt->execute([$card_id, $status]);
        return $stmt->fetch(PDO::FETCH_ASSOC);
    }

    //      ==================      CHECK      IN
==================
    public function checkIn($card_id) {
        $stmt = $this->conn->prepare("
            INSERT  INTO  {$this->table}  (card_id,
checkin_time, status)
            VALUES (?, NOW(), 'IN')
        ");
        return $stmt->execute([$card_id]);
    }

    // ================== CHECK OUT (JADI WAITING)
==================
    public function checkOut($id) {

        $data = $this->getParkirById($id);
```

```
        if (!$data) return false;

        $stmt = $this->conn->prepare("
            UPDATE {$this->table} SET
                checkout_time = NOW(),
                duration                        =
CEIL(TIMESTAMPDIFF(SECOND, checkin_time, NOW()) /
3600),
                fee  =  CEIL(TIMESTAMPDIFF(SECOND,
checkin_time, NOW()) / 3600) * ?,
                status = 'WAITING'
            WHERE id = ?
        ");

        return
$stmt->execute([$this->tarif_per_jam, $id]);
    }

    //          ==================          SELESAI
==================
    public function markDone($id) {
        $stmt = $this->conn->prepare("
            UPDATE {$this->table}
            SET status='DONE'
            WHERE id=?
        ");
        return $stmt->execute([$id]);
    }
}
?>
```

4. Controller/ParkirController.php

```
<?php
require_once __DIR__ . '/../models/ParkirModel.php';
require_once __DIR__ . '/../vendor/autoload.php';

use PhpMqtt\Client\MqttClient;

class ParkirController {

    private $model;

    public function __construct() {
        $this->model = new ParkirModel();
    }

    public function bukaPalang($id) {
        if (!$id) {
            echo "ID tidak ditemukan";
            return;
        }

        $this->model->markDone($id);

        $server = 'broker.hivemq.com';
        $port = 1883;
        $clientId = 'web_' . rand();

        $mqtt = new MqttClient($server, $port,
$clientId);
        $mqtt->connect();

        $mqtt->publish('parking/ayla/exit/servo',
'OPEN', 0);
```

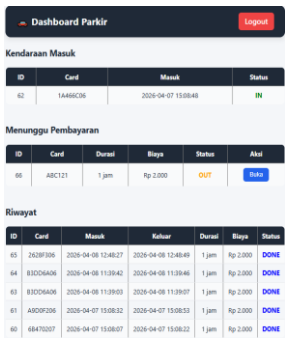
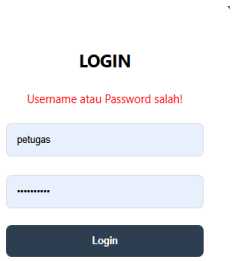
```
                $mqtt->disconnect();

                header("Location: ../index.php");
                exit;
            }
        }
        ?>
```

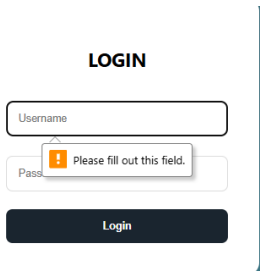
14. TESTING APLIKASI

Berikut merupakan tahapan uji coba atau testing dari beberapa fitur aplikasi yang sudah dibuat dengan menggunakan metode testing Black Box.

1. Tabel Login

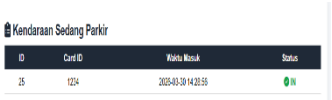
Hasil Pengujian Black Box Testing

No	Skenario Pengujian	Test Case	Langkah Pengujian	Hasil hasil yang diharapkan	Hasil pengujian	Kesimpulan
1.	Login berhasil	Username & password benar	1. Buka login.php 2. Input username & password 3. Klik login	Masuk ke dashboard		PASS
2.	Login gagal	Password salah	1. Input password salah 2. Klik login	Muncul pesan error		PASS

3.	Field kosong	Tidak isi form	1. Kosongkan field 2. Klik login	Muncul “Please fill out this field”		PASS

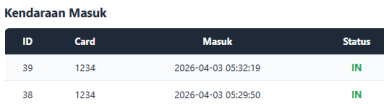
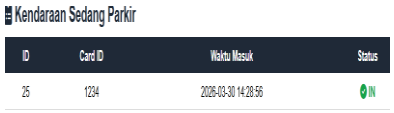
2. Tabel Dashboard

Hasil Pengujian Black Box Testing

No	Skenario Pengujian	Test Case	Langkah Pengujian	Hasil yang Di harapkan	Hasil Pengujian	Kesimpulan
1.	Data kendaraan parkir	Buka dashboard	1. Login ke sistem 2. Buka halaman dashboard	Data kendaraan sedang parkir tampil		PASS
2.	Data menunggu pembayaran	Buka dashboard	1. Login ke sistem 2. Buka halaman dashboard	Data kendaraan muncul di menu menunggu pembayaran		PASS
3.	Riwayat transaksi	Buka dashboard	1. Login ke sistem 2. Buka halaman dashboard	Data riwayat transaksi tampil		PASS

3. Tabel Check-in

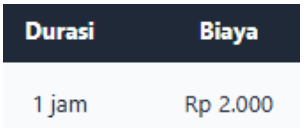
Hasil Pengujian Black Box Testing

No	Skenario Pengujian	Test Case	Langkah Pengujian	Hasil yang Di harapkan	Hasil Pengujian	Kesimpulan
1.	RFID terbaca	Tap kartu RFID	1. Tempelkan kartu RFID ke alat	Sistem membaca card_id dan mengirim ke web	 Kendaraan Masuk ID Card Masuk Status 39 1234 2026-04-03 05:32:19 IN 38 1234 2026-04-03 05:29:50 IN	PASS
2.	Check-in	Kartu belum parkir	1. Tap kartu	Data masuk (status IN) tersimpan	 Kendaraan Sedang Parkir ID Card ID Waktu Masuk Status 25 1234 2026-03-30 14:28:56 IN	PASS
3.	RFID tidak terdaftar	Kartu tidak dikenal	1. Tap kartu	Sistem menolak akses (LCD menampilkan error)		PASS

4. Tabel Check-out

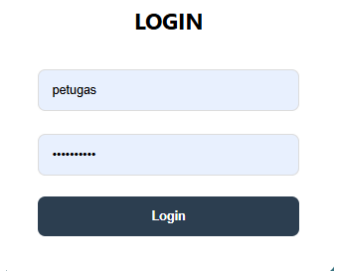
Hasil Pengujian Black Box Testing

No	Skenario Pengujian	Test Case	Langkah Pengujian	Hasil yang Di harapkan	Hasil Pengujian	Kesimpulan
1.	RFID keluar	Tap kartu keluar	1. Tempelkan kartu	Data keluar muncul	Menunggu Pembayaran ID Card Durasi Biaya Status Aksi 69 ABC121 1 jam Rp 2.000 OUT Buka	PASS

2.	Hitung durasi & biaya	Ada data masuk	1. Tap keluar	Durasi & biaya otomatis muncul		PASS
3.	Buka palang	Setelah proses keluar	1. Tap kartu	Palang terbuka otomatis		PASS
4.	Riwayat transaksi	Setelah keluar	1. Cek dashboard	Data masuk ke riwayat (DONE)		PASS

5. Tabel logout

Hasil Pengujian Black Box Testing

No	Skenario Pengujian	Test Case	Langkah Pengujian	Hasil yang Di harapkan	Hasil Pengujian	Kesimpulan
1.	Logout Berhasil	Klik Logout	1. Login 2. Klik logout	Kembali ke halaman login		PASS

15. REVISI

16. ALAMAT URL APK

Localhost/ukk_ayla_parkir/views/index.php

17. LINK GITHUB

https://github.com/Aylapratiwi/Aplikasi-ukk-parkir-rfid-ayla.git